

Ask

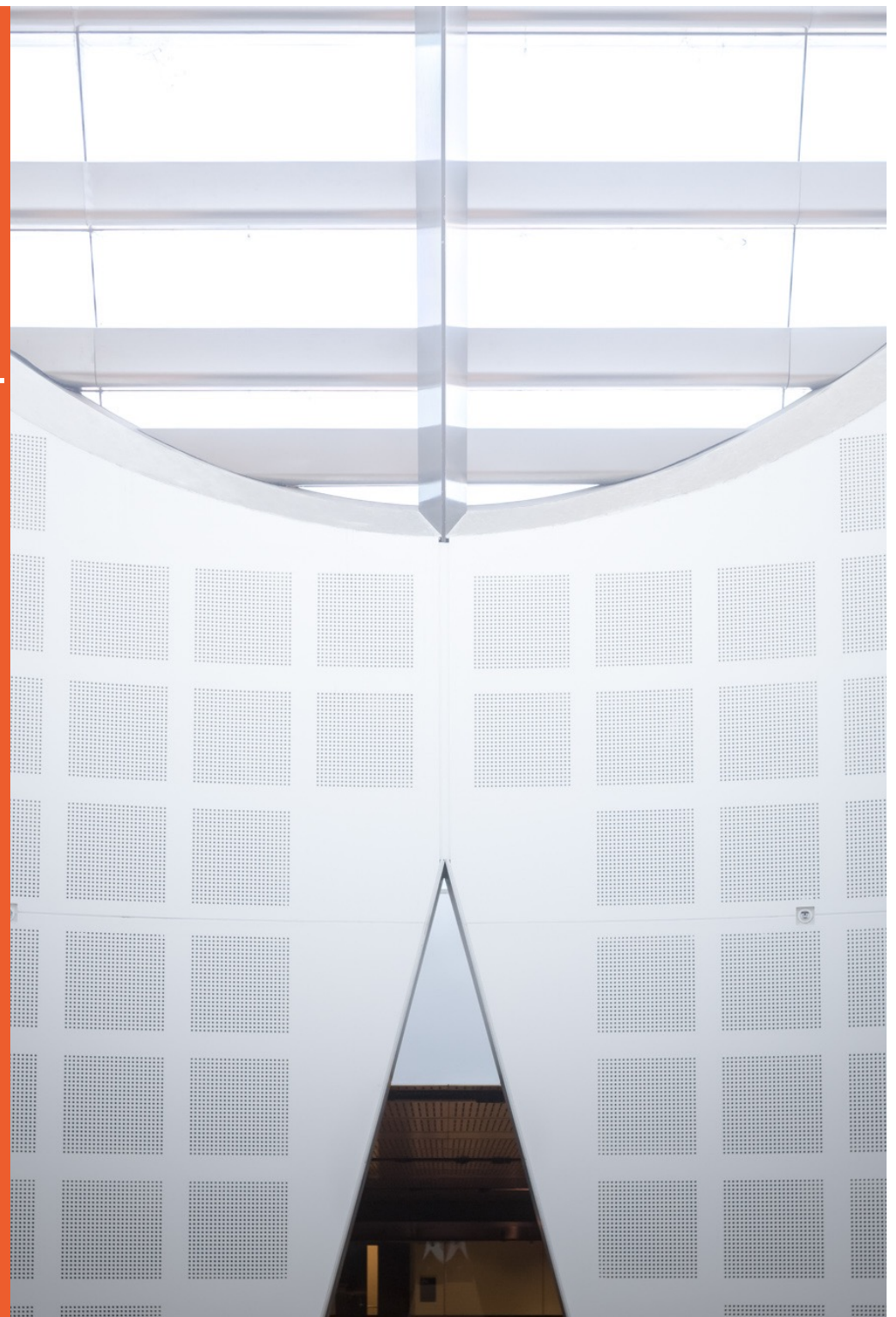


No qualifications needed

Pre-packaged Software – Deploying from the shoulders of giants

Presented by
Dr. Huaming Chen

ELEC3609/9609
Internet Software Platforms
2026 S2



We acknowledge the tradition of
custodianship and law of the Country on which
the University of Sydney campuses stand.
We pay our respects to those who have cared
and continue to care for Country.



THE UNIVERSITY OF
SYDNEY

Overview

✓ **Software Packages**

- ✓ **Components**
 - ✓ **Application Code**
 - ✓ **Configuration**
 - ✓ **Database**
- ✓ **Deployment**
 - ✓ **Reverse Proxy**

✓ **Docker**

- ✓ **Containers & Images**
- ✓ **Volumes & Networking**
- ✓ **Docker Hub**
- ✓ **AWS Elastic Container Services (ECS)**

✓ **Monitoring**

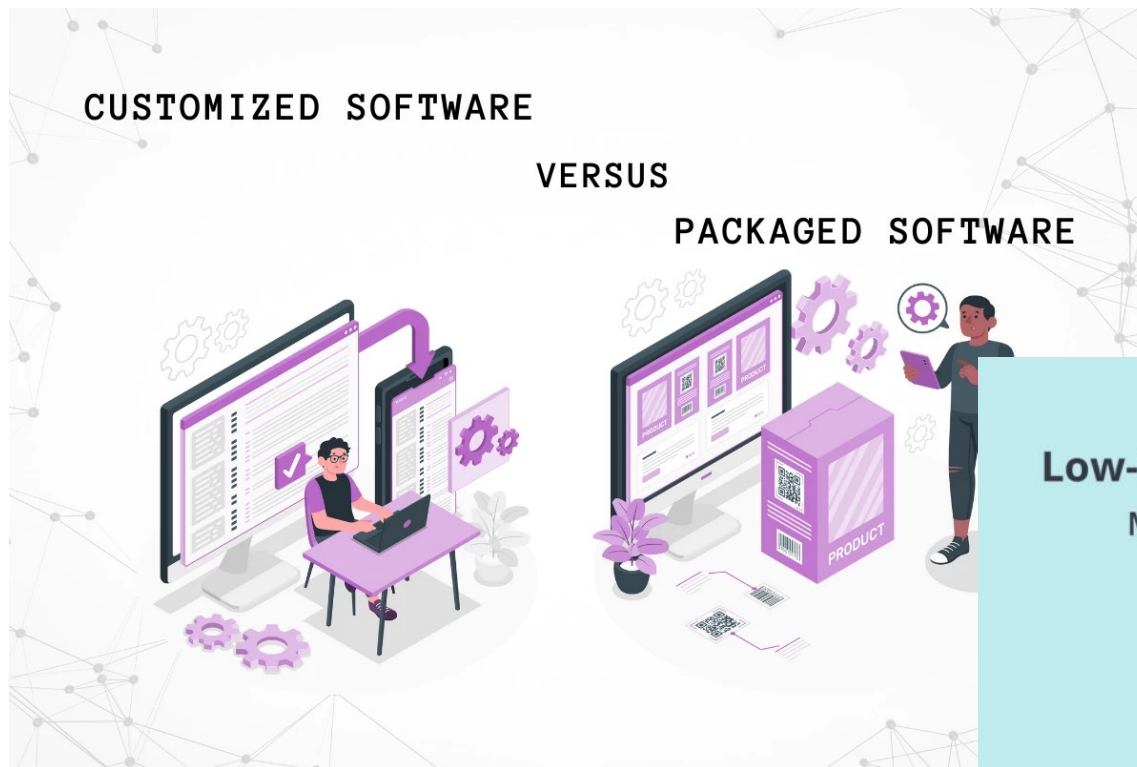
- ✓ **Prometheus**
- ✓ **Grafana**
- ✓ **Elasticsearch ELK**
- ✓ **Nagios**
- ✓ **Sentry**

✓ **Tracking/Analytics**

- ✓ **Google Analytics**
- ✓ **KISSmetrics**
- ✓ **Custom Tracking/ETL**

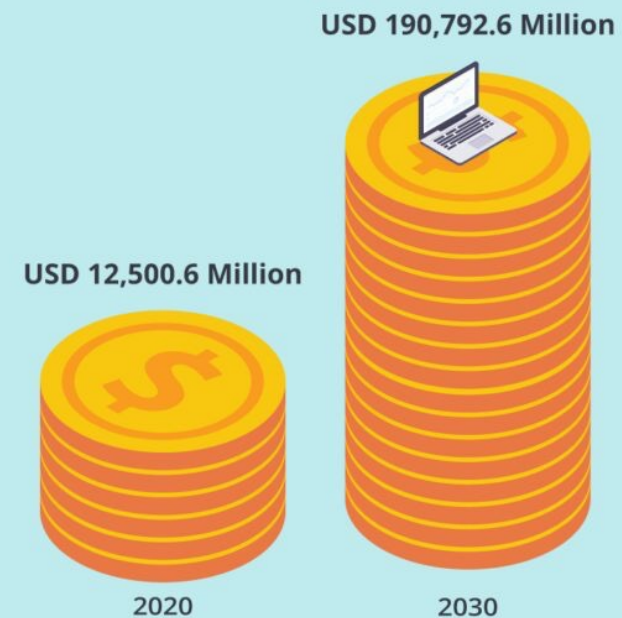
Software Packages

Software Packages



Low-Code Development Platform Market

Market forecast to grow at a CAGR of 31.3%



Software Packages Components

- Prepackaged web software is commonly used to get something up and running quickly. Examples include:
 - **Wordpress**: Content Management System (CMS)
 - **vBulletin**: Internet forum software
 - **MediaWiki**: Open-source wiki platform
 - **Phabricator**: Project management tool



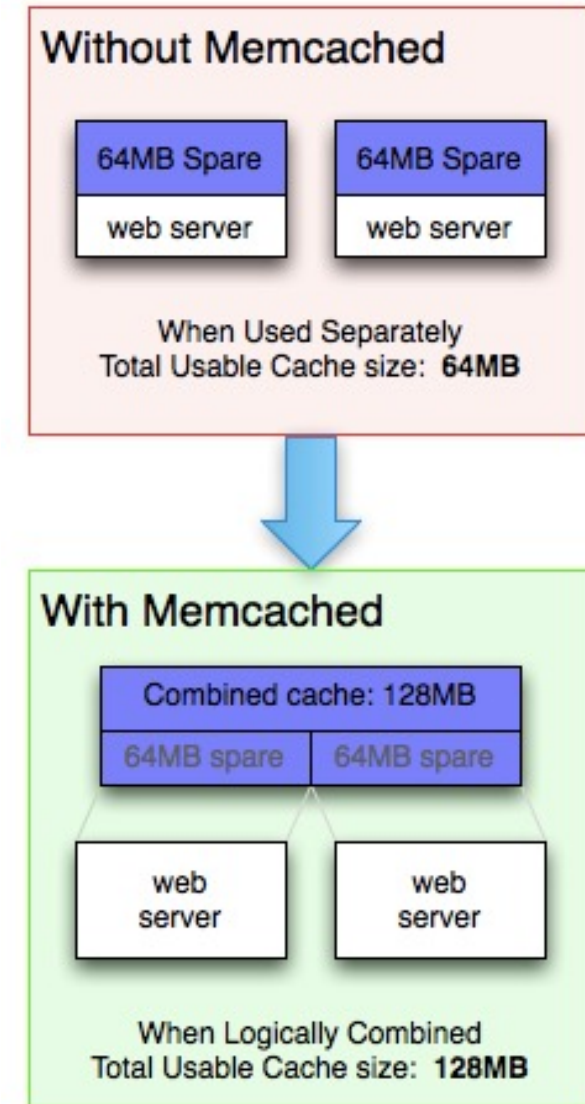
vBulletin®



MediaWiki

Software Packages Components

- Just like any other website, these packages typically consist of:
 - application code
 - configuration files
 - database
- The prepacked software provides application code, but it is up to the deployer to configure the application and infrastructure.
 - **Memcached**

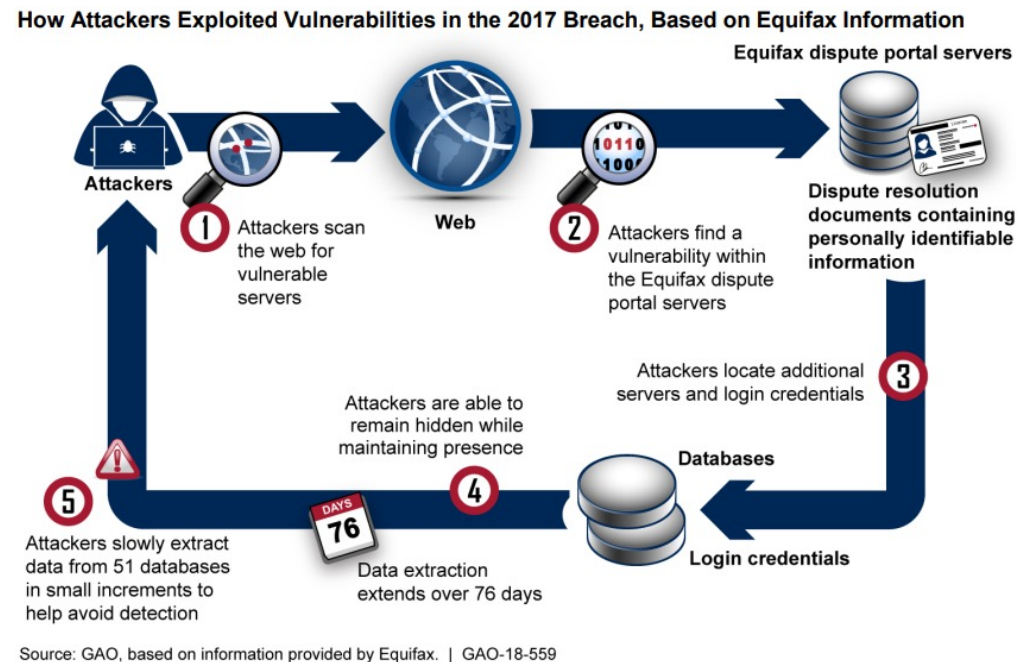


Software Packages Security

- Any perceived risks?
 - Inheriting not just the main application code, but also all of its dependencies
- Dependency management
 - Libraries & frameworks
 - Plugins / extensions
 - Container images
 - System packages
- Supply chain security
 - Supply Chain Attacks
 - Outdated Dependencies
 - Plugin/Extension Ecosystem
 - Trust in Docker Hub Images

Software Packages: Application Code

- It can be dangerous to blindly run other people's code, especially when it is old and complex.
- Questions to ask:
 - What is it written in?
 - Is the code-base mature?
 - How active is development on the project?
 - Does the software have a history of security vulnerabilities?



Software Packages: Application Code

- **Identify other users**
 - If large companies have adopted the software, it is likely they have reviewed it before deciding.
- **Identify potential support avenues**
 - Determine if support is available through the software supplier. Do they provide direct support? How much does it cost?
- **Check for the frequency of commits**
 - If the software is open-source, take a look the activity of their code repository. More frequent and recent, the more active development.
- **Scan for vulnerabilities**
 - Depending on the language and tooling, static source code vulnerability scanner.
- **Search for known vulnerabilities**
 - Check sites disclosing existing vulnerabilities - cvedetails.com

Software Packages: Application Code

- npm audit: <https://docs.npmjs.com/cli/v9/commands/npm-audit>
- pip-audit: <https://pypi.org/project/pip-audit/>
- OWASP Dependency-Check: <https://owasp.org/www-project-dependency-check/>
- Snyk Vulnerability DB: <https://security.snyk.io/>

Software Packages: Configuration

- While it is good practice to release software that is secure-by-default, many software packages come with insecure configuration.
- It is critical to review all configuration options prior deployment.
- Question to ask:
 - What configuration options are available?
 - Are the default values for these options secure?
 - Are there any features enabled that you're not using?
 - Can you disable these features?

Software Packages: Configuration

- **Identify all the configuration options**
 - Be sure to read the documentation and view example configurations.
- **Search the Internet for example configurations**
 - Generally configurations are readily available online. Don't blindly trust these. Evaluate the configuration.
- **Identify all features and components**
 - Usually prepackaged software is general purpose. This means while it has the functionality required for your application, it may also offer other features.
- **Disable unwanted features**
 - Reduce complexity and surface area for possible attacks.

Software Packages: Configuration



Download Our Free Benchmark PDFs

The CIS Benchmarks are distributed free of charge in PDF format for non-commercial use to propagate their worldwide use and adoption as user-originated, de facto standards.



CIS Benchmarks are the only consensus-based, best-practice security configuration guides both developed and accepted by government, business, industry, and academia.



Software Packages: Database

- It is usually the responsibility of the deployer to configure the database required by pre-packaged software.
- Care must be taken while doing this.
- Questions to ask:
 - Does the package need to have its own database or can/will it share?
 - What DB permissions are required?
 - If the application is vulnerable, will DB compromise lead to system compromise?

Software Packages: Database

- **Identify database requirements**
 - Does the software create a single database for all of its tables? Or does it create multiple databases?
- **Evaluate database permissions**
 - It is best practice to give the minimum set of permissions to an application. A good approach is to start by giving the minimal set of access and extending when necessary.
- **Evaluate the scope of DB compromise**
 - If a malicious user discovers an SQL injection in our application, they may be able to run arbitrary commands on our database. Important to update database as well.

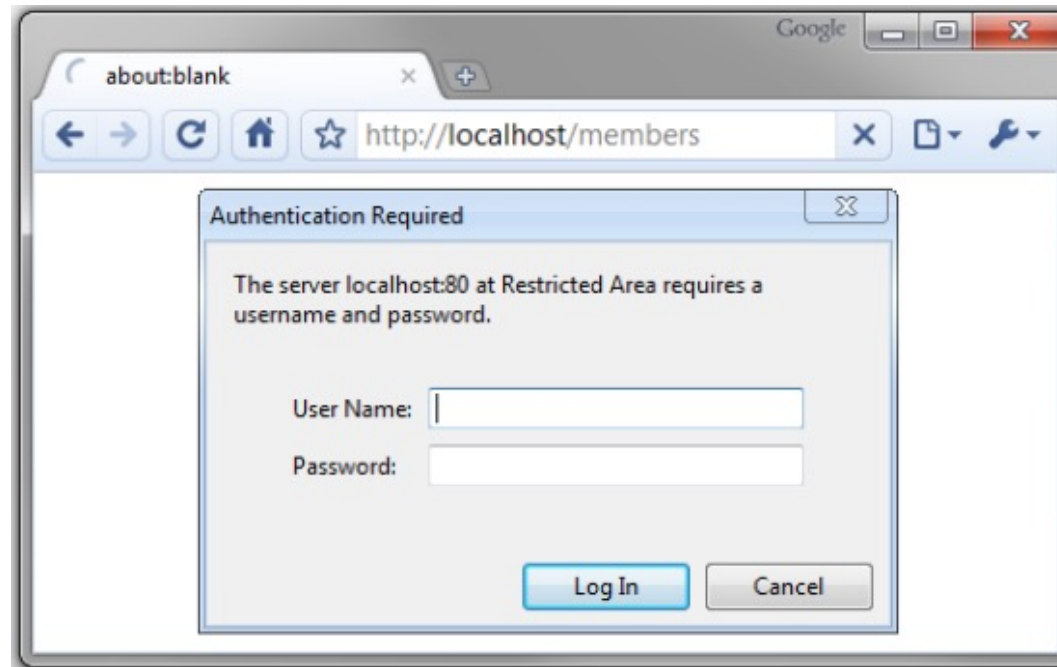
Deployment of Software Packages

- Software packages can come with a web/HTTP interface.
- We don't want this interface *directly* exposed to the Internet, we want our own layer that we have control over in between.
- A reverse proxy (Nginx, Apache) or load balancer can enable us finer control over which requests can reach the application.
- Especially important if the application exposes an admin or control interface without authentication.

Reverse Proxy

- Intercepting incoming web requests with a reverse proxy (RP) can provide many advantages:
- **Scaling**
- We can increase the number of concurrent connections by using the RP as a load balancer. We can run many instances of the application and scale horizontally. Database is still a bottleneck to share state.
- **Request Authentication**
- If our application is an internal service, we can perform basic authentication to keep anonymous web users from accessing it.
- Malicious user can't access the application if blocked by RP.
- **Web Application Firewall (WAF)**
- A WAF can inspect incoming requests and reject them if they appear to be attack vectors

HTTP Basic Authentication



- HTTP Basic Authentication requires HTTP requests to include a username and password to be considered authorised.
- If the HTTP requests arrives without a valid username and password, a 401 Unauthorized response is served.

HTTP Basic Authentication - Mechanics

- Server Side:
 - 1. Server responds to initial request with a 401 Unauthorized HTTP response, containing a HTTP header:
WWW-Authenticate: Basic realm="User Visible Realm"
- Client Side:
 - 1. Prompts user for username/password
 - 2. Combines username and password with a single colon:
"Aladdin:OpenSesame"
 - 3. Encodes string in RFC2045-MIME variant of base64.
 - 4. Sets HTTP request *Authorization* header to base64-encoded credentials:
Authorization: Basic QWxhZGRpbjpPcGVuU2VzYW11

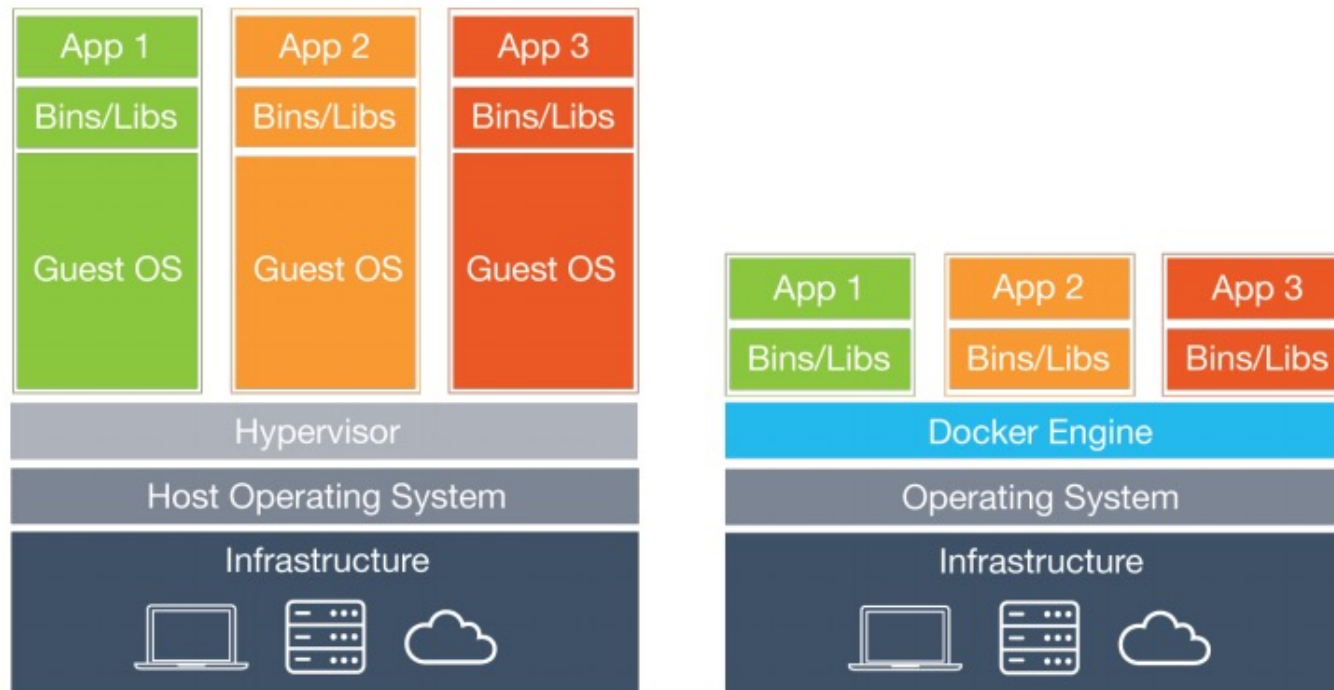
Web Application Firewall

- A Web Application Firewall (WAF) applies rules and/or filters to incoming HTTP requests. A WAF will try to identify HTTP requests that appear to contain exploitation code and reject them.
- All requests should go through the WAF, so it usually sits at the edge/web layer in front of your application.
- A WAF is only as good as the rules it evaluates. Keep the rule/filter sets up to date.

Docker

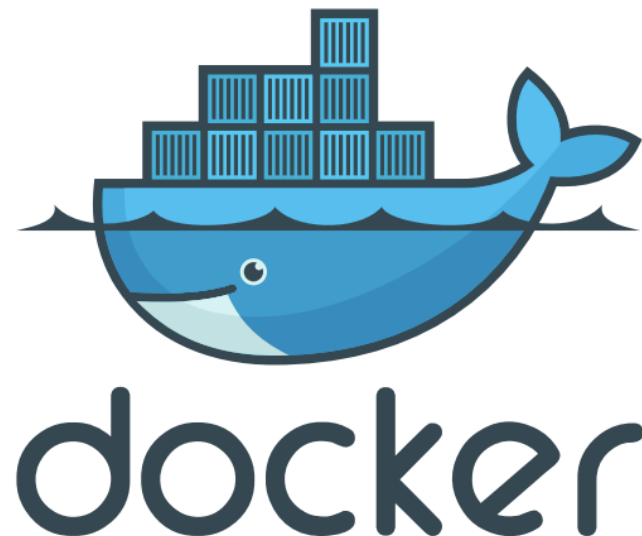
Virtual Machines vs Linux Containers

- A hypervisor enables the execution of virtual machines inside a host operating system.
- Linux containers remove the need to install a guest operating system, allowing multiple isolated Linux systems to run on a single host operating system.



Docker

- Docker is an open-source project to automate the deployment of applications inside Linux containers.
- It allows for easy deployment of packaged applications, including any necessary dependencies.
- Multiple services/containers can be combined using the tool: Docker Compose



Docker: Containers

- Instead of running virtual machines, Docker allows us to run *containers*.
- A Docker container can be thought of as a minimalistic Linux host.
- We can install any application that we like into the container, and the container itself can be (almost) any distribution of Linux.
- A Docker container is created based off a *Docker image*. An image file describes the base operating system, and any modifications that need to be made, such as dependencies.

Docker: Image

- Sample Docker image that specifies:
 - use Ubuntu 18.04 (Xenial) as the base image.
 - executes make in the */app* (mounted filesystem).
 - copy all files from the */app* directory.
 - invoke Python with the entry point of our application.
- FROM ubuntu:18.04
- RUN make /app
- COPY . /app
- CMD python /app/app.py

Docker: Volumes

- Docker containers are good for running applications, but they are not good for storing data. Containers do not maintain state. Typically, we mount *volumes* into Docker containers to allow for persistence.
- If we were creating a Docker container for a database, such as MySQL, the data directory for the application should be stored in an attached volume.
- The mounted volume is a folder on the host machine and exists outside the container. We can now destroy the container without losing any data.

Docker: Networking

- Docker containers can be *linked* to each other and have nominated ports forwarded from the host OS to the container.
- If we have a web application, such as Wordpress, we can have one container for the application and a separate container for the MySQL database.
- By linking the containers, the Wordpress application can talk to the MySQL container directly. No other containers or system are allowed to communicate with the database unless specified directly via Docker.
- This segmentation significantly improves security and ease of deployment.

Docker Hub

- Why re-create the wheel? We don't need to build our own Docker containers, we can stand on the shoulders of the giants who have come before us.
- The Docker project has a service called Docker Hub, which contains pre-built Docker images. These can be used as a starting point for building services.
- Many large software companies and projects have released Docker Hub images for their software projects.
- **Security is a factor to consider here.**

Cloud – Container Orchestration

- When running containers in production, it is quite common to have container fleets of 10s-1000s. How do we manage these?
- Google offers Kubernetes Engine (GKE), an open-source cluster management system, for deploying and managing containerised applications on Google Cloud. You can also use Kubernetes outside of Google Cloud.
- AWS offers the Elastic Container Service for deploying Linux containers within the AWS ecosystem.

Monitoring

Monitoring

- When running a large-scale website, it is essential to monitor everything.
- Facets we want to consider:
 - User Interaction
Who clicks on what?
 - User Analytics
What local time do users usually come online?
 - HTTP Error Rate
What pages are throwing 500s and impacting users?
 - Infrastructure Metrics
How is our infrastructure performing? Can we handle 2x load? 10x?

System Monitoring

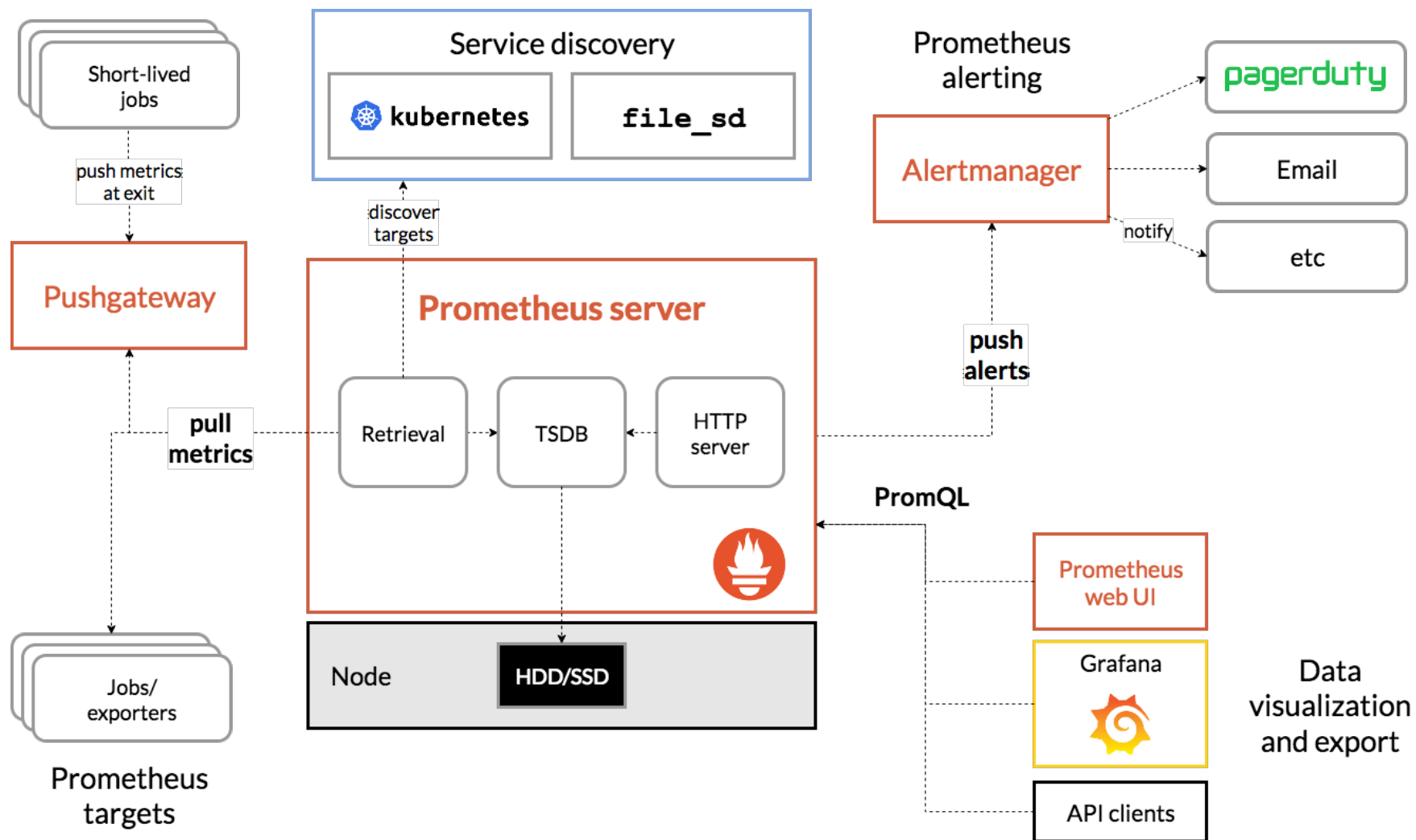
- There are many tools and techniques available for monitoring websites metrics at the system and application level.
- In following slides we will discuss:
 - Prometheus: Metrics
 - Grafana: graphing above metrics + dashbaords
 - Elasticsearch, Logstash, Kibana (ELK): Logging
 - Nagios: Infrastructure monitoring
 - Sentry: Error detection

Prometheus

- Prometheus is an open-source system for storing and graphing highly dimensional data and metrics.
- Efficiently stores metrics in a custom format and scales through functional sharding and federation. Data is scraped from monitored services over HTTP, rather than the applications publishing metrics to Prometheus.
- Developed at SoundCloud originally, now a member of the Cloud Native Computing Foundation.



Prometheus



Grafana

- Grafana allows us to visualise, query and alert on key metrics, regardless of where they are stored. Grafana can have multiple backend metrics systems.



Grafana

Demo

- **System Metrics:**
 - CPU usage
 - Memory usage
 - Disk space and I/O
 - Network traffic and bandwidth
 - Load averages
 - System uptime
- **Application Metrics:**
 - Response times
 - Throughput
 - Error rates
 - Requests per second
 - Latency
 - User activity metrics

etc

Elasticsearch

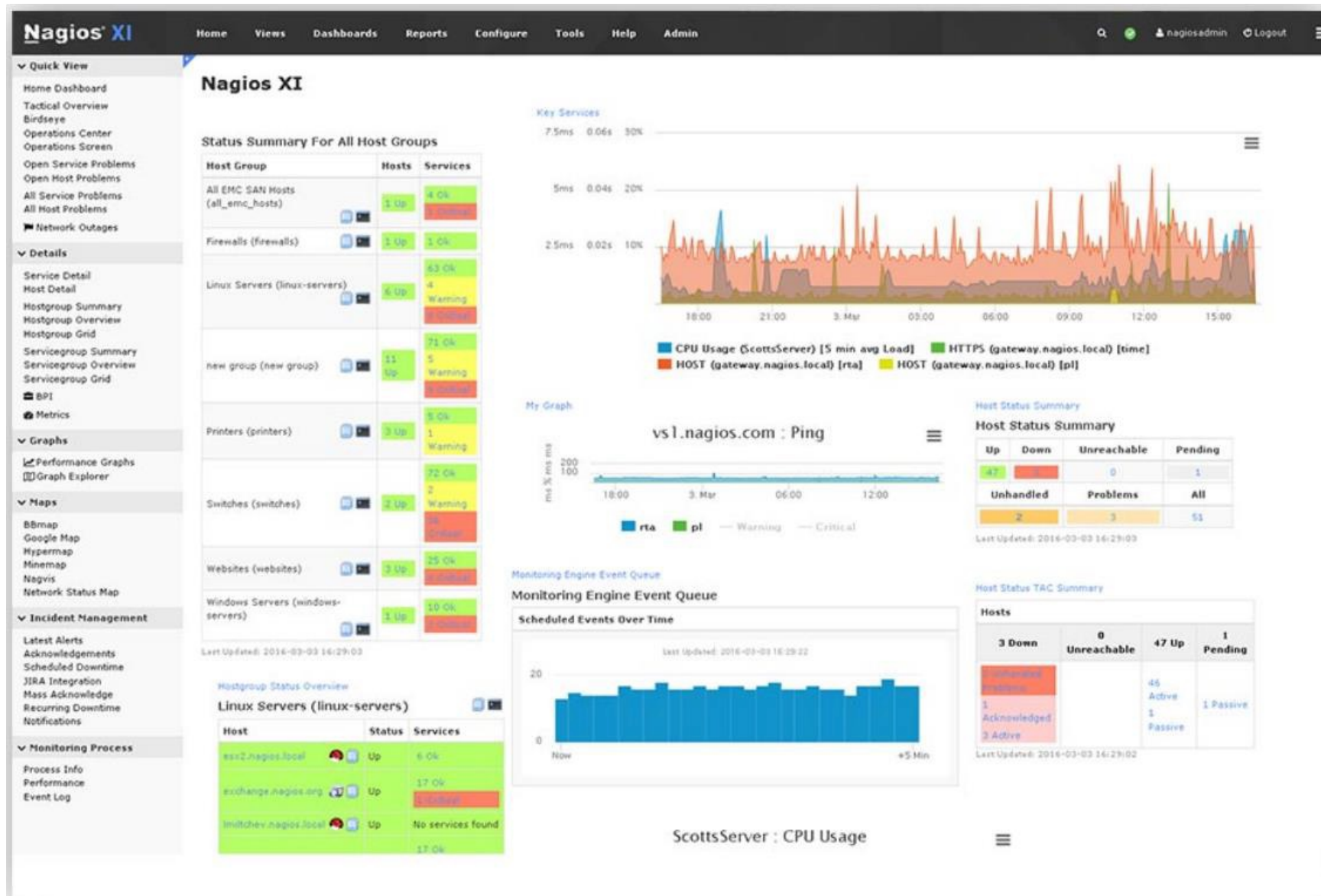
- Once our website reaches a certain size, manually looking at logs on each host is not feasible.
- The ELK stack (Elasticsearch, Logstash, Kibana) is a commonly used industry pattern for ingesting and querying large volumes of logs (application, web access/error, etc) in a centralised location.
- Kibana is an open-source data visualisation plugin for Elasticsearch. It offers a simple interface for querying Elasticsearch (the service that has our logs).



NAGIOS

- Nagios is an open-source “industry standard” for monitoring infrastructure.
- Many custom metrics can be configured to monitor:
 - Network transfer rates
 - Server performance
 - Application metrics
- Nagios has integrations that allow it to send email, integrate with chat programs and trigger alerts if metrics are exceeded.

System Monitoring



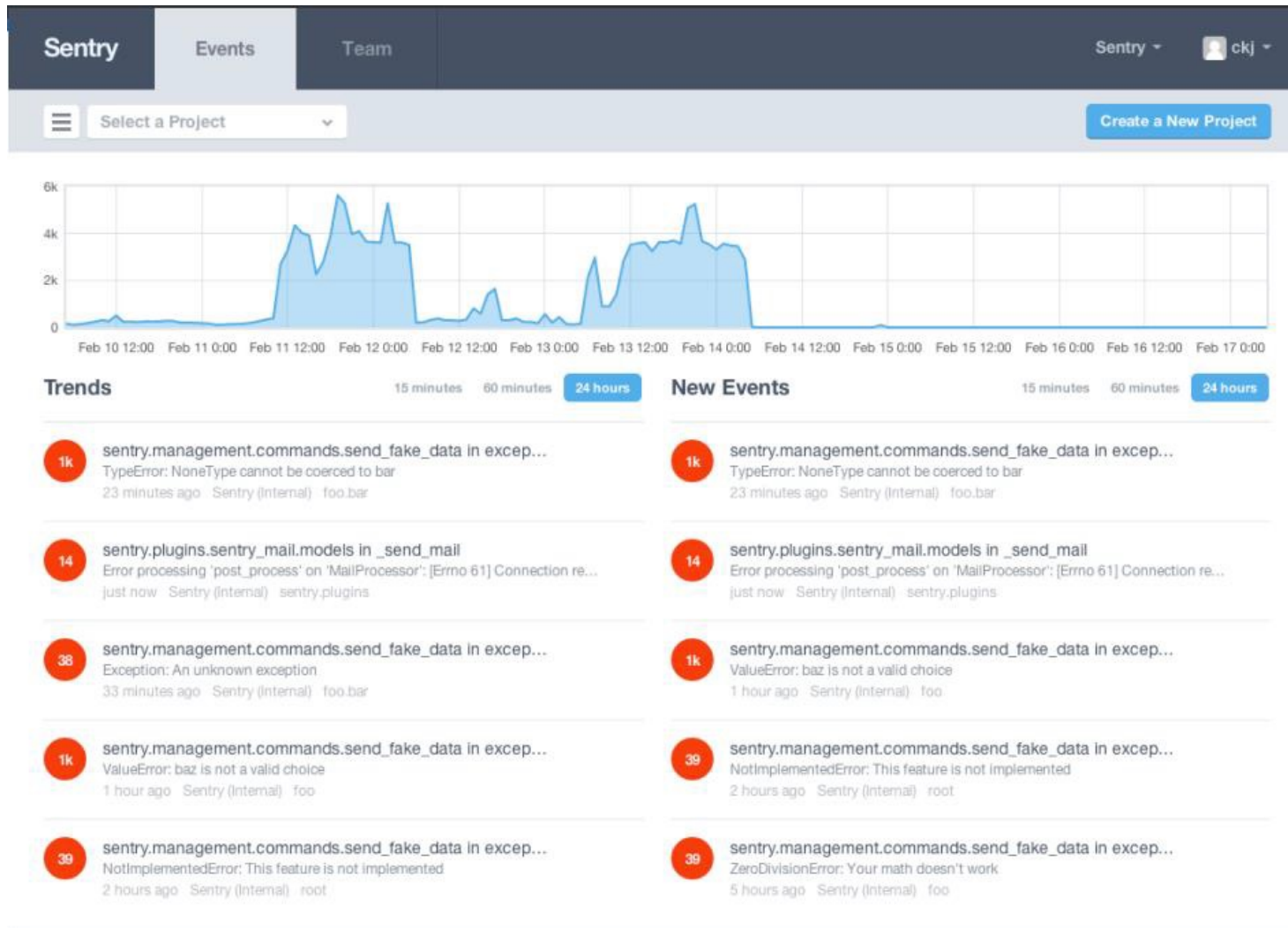
Sentry

- Sentry collects errors from your website, both front-end and back-end, and consolidates and aggregates them into a single feed of events.
- It allows for real-time monitoring of errors experienced while using your website, and is a critical piece of infrastructure for large scale websites.



"Stop hoping your users will report errors"

Sentry



Sentry

Events

Team

Sentry 

 Sentry (Internal) 

 TypeError: NoneType cannot be coerced to bar

Search query or message id

 sentry.management.commands.send_fake_data in exception
TypeError: NoneType cannot be coerced to bar
26 minutes ago foo.bar  

 The system marked this event as a regression — just now
 chris@getsentry.com marked this event as resolved — 2013-01-28

4 Newer Event

Event at Feb. 16, 2013, 2:23 p.m. UTC

Older Event 

Exception

Stacktrace

User

Additional Data

Versions

Exception

Types:	TypeError
Value:	NoneType cannot be coerced to bar
Location:	sentry/management/commands/send_fake_data.py in exception , line 39

Stacktrace

(most recent call last)

Raw

sentry/management/commands/send_fake_data.py in exception  

```
34. # return client.capture('Query', query=queries.next(), engine=engine.next(), time_spent=duration, data={'logger': loggers.next(), 'site': 'sql'})
35.
36. def exception(client):
37.     timestamp = datetime.datetime.utcnow() - datetime.timedelta(seconds=timestamps.next())
38.     try:
39.         raise exceptions.next()
40.     except Exception:
41.         email = emails.next()
42.         return client.capture('Exception', data=[
43.             'logger': loggers.next(),
44.             'site': 'web',
```

Aggregate

Tags

Similar Events

AGGREGATE DETAILS

Status: Unresolved

First Seen: Jan. 19, 2013

Reopened At: 16 hours ago

Last Seen: 24 minutes ago

This event is publicly visible. [Link](#)

ACTIONS

Remove Event Data

LEVEL

1.0k error

LOGGER

1.0k foo.bar

SERVER NAME

1.0k Chris-MacBook-Pro.local

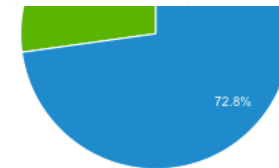
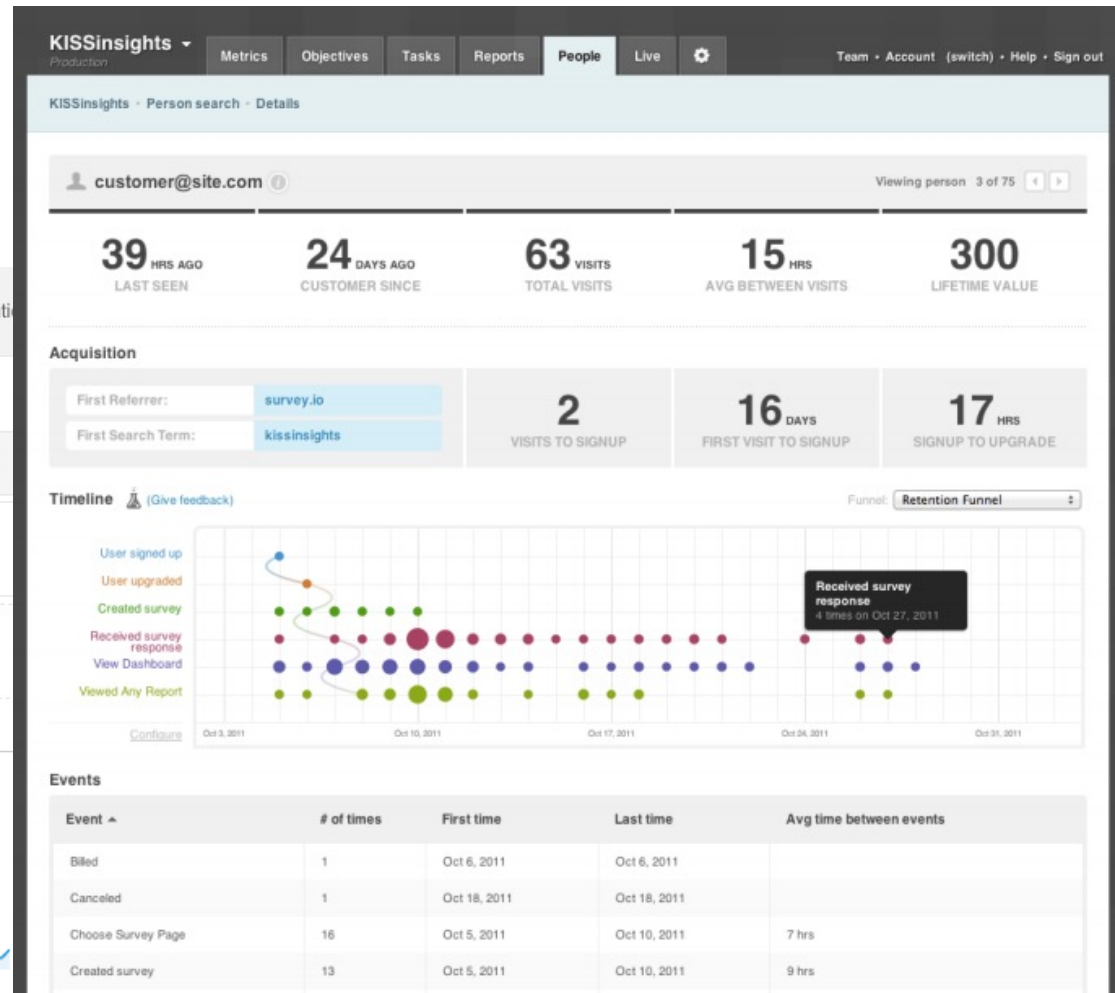
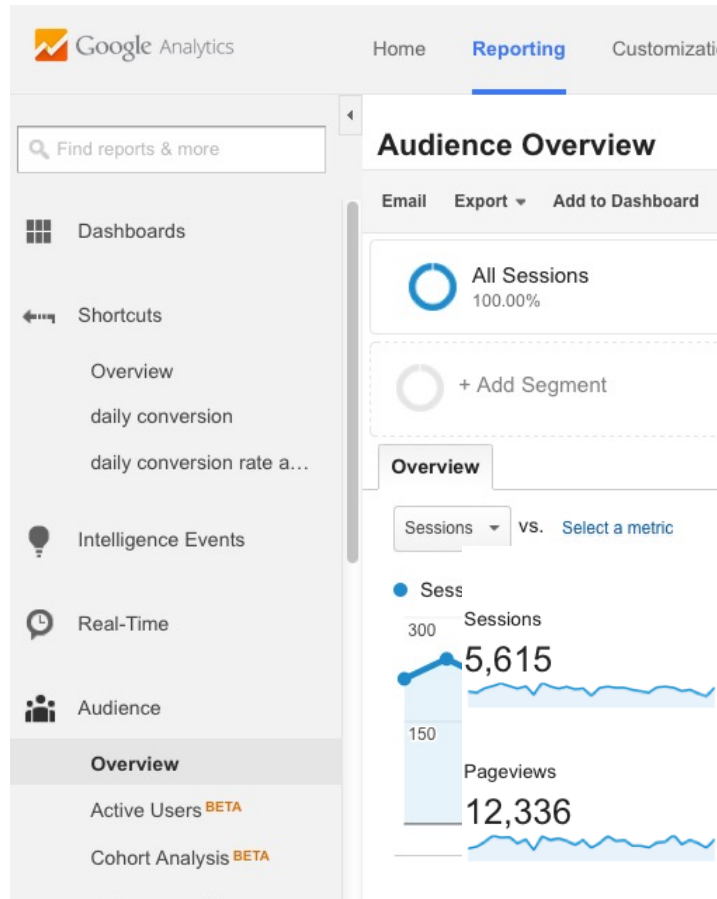
SITE

1.0k web

Some others

- Google Analytics
- KISSmetrics

Some others

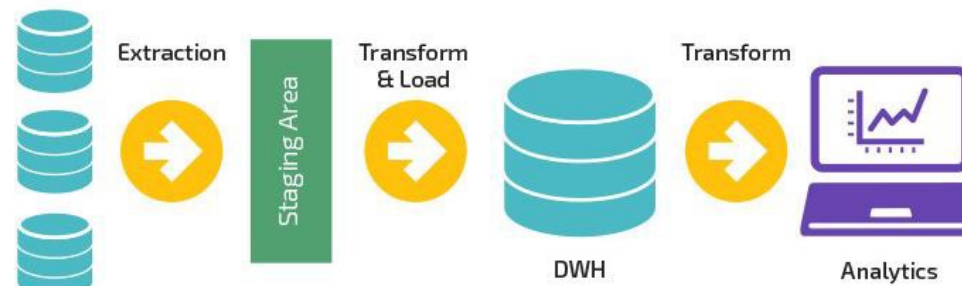


Custom Tracking/Analytics

- You may find that user tracking systems readily available on the market are too heavy (performance) or do not meet your needs. In situations like these, you may need to build out the beginnings of a data warehouse solution.
- Database schemas are generally optimised for user performance, not for analytics. Calculating a user's balance by summing their deposits is relatively straight forward. Calculating the balance for all users is less-bounded.
- Transforming the database into a format that is better optimised for queries, or importing the data into another service, are the most common ways of dealing with this.

ETL

- When building a custom tracking system, a common approach is to use the **Extract, Transform, Load (ETL)** process:
- **Extract:** Extract the data from various sources
 - User clicks from JavaScript, web access logs, database changes
- **Transform:** Transform the data into a consistent format or structure.
- **Load:** Load the transformed data into a final application.



MapReduce

- If we need to perform very computationally intensive analytics tasks across a large data set, a useful technique can be MapReduce.
- The approach is to perform the data analytics in a distributed sense, splitting over two different steps.
 - Map: Performs filtering and sorting.
 - Reduce: Consolidates data into a summary format.

**You reached the end of the lecture.
Thank you and see you next week.**